COMPUTER NETWORKS

PROJECT REPORT

BBR vs CUBIC CONGESTION CONTROL ALGORITHMS

Name	Roll Number
V DIVYADARSHINI	CB.SC.U4CYS24014
KARNIKA V S	CB.SC.U4CYS24028
SHRUTHI PAVALAVEL	CB.SC.U4CYS24054
PRAVEENA P	CB.SC.U4CYS24064

ABSTRACT

Congestion control affects how senders share a network link and balance throughput, delay, packet loss, and fairness. This project focuses on TCP BBR versus TCP CUBIC and also reviews TCP Reno, TCP Westwood+, QUIC-DC, PowerTCP, and DNCCQ-PPO. Experiments used Linux network namespaces, Mininet, iperf3, traffic control, QUIC tools, and ns-3.

In the recorded shared-link tests, CUBIC received 8.65 Mbps and BBR 1.86 Mbps on a 10 Mbps link (Jain's fairness index: 0.643). On a 5 Mbps link, they received 3.82 Mbps and 1.77 Mbps, respectively. In a separate 25 Mbps setup, QUIC-DC reached 23.449 Mbps with low measured loss. The Reno–Westwood+ results still need validation, and the DNCCQ-PPO reinforcement-learning loop was not completed. The results show that performance depends on link capacity, RTT, buffering, queue discipline, and competing traffic.

Keywords: *congestion control, TCP CUBIC, BBR, QUIC-DC, TCP Reno, Westwood+, PowerTCP, DNCCQ-PPO, RTT, goodput, fairness, bufferbloat*

1. INTRODUCTION

1.1 Background

Every sender on a shared network path must decide how fast to transmit. Sending too slowly wastes capacity, while sending too fast fills router buffers, increases delay, and causes packet loss. Congestion control algorithms regulate this behaviour, and their design determines how throughput, queueing delay, packet loss, and fairness between competing flows are balanced.

Different algorithms use different congestion signals. Loss-based algorithms react to dropped packets, bandwidth-aware algorithms estimate the available rate from acknowledgements, delay-based algorithms watch queueing delay, and learning-based algorithms choose actions from observed network conditions. Their behaviour therefore varies with link capacity, round-trip time, buffer size, queue discipline, and competing traffic.

1.2 Objectives

The objectives of this project are:

- to compare TCP BBR and TCP CUBIC on a shared bottleneck link in terms of throughput and fairness;
- to review TCP Reno, TCP Westwood+, QUIC-DC, PowerTCP, and DNCCQ-PPO as bandwidth-aware, delay-based, datacenter, and learning-based approaches;
- to evaluate each algorithm using goodput, round-trip time, packet loss, fairness, and flow completion time; and
- to report the limitations of each experiment clearly, so that results are not over-generalised.

1.3 Algorithms Considered

- **TCP BBR:** estimates the bottleneck bandwidth and minimum round-trip time, then paces packets using that path model instead of treating every loss as a congestion signal. In the BBR–CUBIC tests, its share depended on how it competed with the loss-based flow.
- **TCP CUBIC:** grows its congestion window using a cubic function of the time since the last loss. It is designed for efficient bandwidth use, including on high-bandwidth paths, although aggressive probing can build queues and affect fairness when flows compete.

- **TCP Westwood+:** estimates available bandwidth from the ACK arrival rate and uses the estimate with the minimum RTT to set a recovery window after congestion. Its results depend on the quality of ACK timing and bandwidth estimates.
- **QUIC-DC:** compares sender departure spacing with receiver arrival spacing to estimate queueing delay. When the estimate crosses a configured threshold, it reduces sending aggressiveness to limit queue growth while maintaining goodput.
- **PowerTCP:** targets datacenter traffic. It combines a throughput-to-delay objective with queue-change feedback so that sending rates can respond quickly when multiple flows compete for switch buffers.
- **DNCCQ-PPO:** applies Proximal Policy Optimization (PPO) to choose congestion-control actions from changing network conditions such as RTT and bandwidth. In this project, the environment and telemetry worked, but the complete state/action training loop remained incomplete.

1.4 Project Scope

The main comparison in this report is BBR versus CUBIC on a shared bottleneck. Westwood+, QUIC-DC, PowerTCP, and DNCCQ-PPO broaden the study to bandwidth-aware, delay-based, datacenter, and learning-based approaches. Because their test environments differ, their results are discussed separately rather than treated as one common benchmark.

2. METHODOLOGY

2.1 Research Approach

We reviewed the supplied source files, scripts, configurations, logs, CSV summaries, and graphs. Reported averages are used where trials are available. Results flagged by the experiment files are marked as preliminary.

2.2 Algorithms Covered

Algorithm	Family	Primary signal	Control idea	Intended setting
TCP Reno	Loss-based AIMD	Packet loss / duplicate ACKs	Increase cwnd by about 1 MSS per RTT in congestion avoidance; halve cwnd on congestion.	Legacy baseline and clean wired paths.
TCP CUBIC	Loss-based; cubic window growth	Packet loss and time since last loss	Uses $W(t) = C(t - K)^3 + Wmax$ to probe bandwidth.	General-purpose TCP, bulk transfer and high-BDP paths.
BBR / BBRv2	Model-based, rate-paced	Bottleneck bandwidth and minimum RTT estimates	Paces packets based on estimated delivery capacity and cycles through probing states.	High-bandwidth and long-RTT paths; fairness depends on coexistence.
TCP Westwood+	Bandwidth-aware TCP	ACK arrival-rate estimate and loss	On congestion, sizes cwnd from estimated bandwidth × minimum RTT.	Potentially useful when losses are not all congestion losses.
QUIC-DC	Delay-based QUIC congestion control	One-way queueing-delay estimate (OWQD)	Triggers early backoff at configured delay thresholds; uses a bandwidth estimate.	Latency-sensitive and interactive traffic.
PowerTCP	Datacenter / telemetry-oriented	Network power and queue-length gradient	Adjusts sending rate rapidly using fine-grained queue feedback.	Datacenter fabrics, incast, and short flow completion time.
DNCCQ-PPO	Reinforcement-learning-based QUIC CC	Dynamic state including RTT, loss, bandwidth and queue behaviour	PPO policy is intended to select sending-rate or congestion-window actions.	Changing network conditions; requires tight transport-stack integration.

2.3 Evaluation Metrics

Metric	Meaning in this project
Goodput / receiver throughput	Useful application payload delivered per unit time, reported in Mbps or Gbps. When a source says “throughput” rather than “goodput”, the report retains that label.
Round-trip time (RTT)	Time for a packet to travel to the peer and for the acknowledgement to return. Increased RTT can indicate queueing delay in addition to base propagation delay.
Packet loss rate	Percentage of transmitted packets reported dropped in the experiment. Loss values are configuration-dependent.
Jain’s Fairness Index	For flow rates x_i, $J = (\Sigma x_i)^2 / (n\Sigma x_i^2)$. A value closer to 1 indicates a more even allocation among n flows.
Flow completion time (FCT)	Elapsed time to complete a flow. Particularly important for short RPCs and datacenter workloads.
Retransmissions and queueing	Supporting diagnostics used to help interpret packet delivery behaviour, queue pressure, and flow-level throughput.

2.4 Test Environments and Experimental Inputs

Workstream	Environment / tools	Recorded configuration
BBR vs CUBIC	Linux network namespaces, virtual Ethernet links, tc traffic shaping, iperf3, kernel socket diagnostics (ss).	Condition A: 10 Mbps bottleneck, 20 ms configured delay. Condition B: 5 Mbps bottleneck, 50 ms configured delay. Solo tests also recorded.
Reno vs Westwood+	Ubuntu/WSL2, Mininet 2.3.0, Open vSwitch, iperf3 3.9.	25 Mbps host links, 25 ms configured delay per host link, 420-packet queue, three 20-second trials per algorithm.
QUIC-DC	Python experiment and plotting pipeline; summary CSV and per-configuration time-series.	25 Mbps bottleneck, 50 ms base RTT, 100 MB transfer; queue buffers of 0.5, 1, 2 and 4 BDP; delay thresholds 10%, 20%, 50% and 80%.
PowerTCP	ns-3 datacenter simulator and supplied PowerTCP example configurations/results.	Fat-tree/datacenter configurations, 25 Gbps server-to-ToR links and 100 Gbps core links in the presentation's summary; fairness and workload artefacts supplied.
DNCCQ-PPO	Mininet, Open vSwitch, Redis pub/sub, XQUIC, Python training code.	Dynamic bandwidth 10–90 Mbps and RTT variation 7–37 ms were successfully published; full policy loop was blocked by missing expected state messages.

2.5 Experimental Procedure

1. Review each experiment's README, scripts, configuration, and logs.
2. Configure the topology, bandwidth, delay, and buffers in the relevant testbed.
3. Run solo baselines and shared-flow tests where available; save raw outputs.
4. Compare the metrics available for each experiment and generate plots from the recorded data.
5. Check results against the logs and flag counter, algorithm-label, or replication issues.

Comparability note: The seven algorithms were not all tested under identical network conditions or a single shared implementation framework. Their values are best read as experiment-specific evidence, not as a universal leaderboard.

3. IMPLEMENTATION

3.1 BBR–CUBIC Coexistence Experiment

BBR and CUBIC were tested as solo flows and then together over a shared bottleneck. The experiment used iperf3, Linux network namespaces, tc for bandwidth and delay, and ss to inspect socket state. The two shared conditions were 10 Mbps with 20 ms configured delay and 5 Mbps with 50 ms configured delay.

Solo throughput was similar, but CUBIC received more bandwidth in both shared tests. This is consistent with CUBIC's loss-driven growth competing with BBR's pacing in the tested FIFO queues; it should not be generalised to every configuration.

3.2 TCP Reno and TCP Westwood+

A two-host Mininet test compared Reno and Westwood+ across three timed trials per algorithm. The CSV contains byte counters and receiver throughput, but the logs flag mismatched counters and missing congestion-control labels. The averages are therefore preliminary, not proof that Reno is generally better.

3.3 QUIC-DC Delay-Based Control

QUIC-DC estimates queue growth by comparing packet departure spacing at the sender with arrival spacing at the receiver. When the estimated one-way queueing delay crosses a configured threshold, it adjusts the congestion window using estimated bandwidth and minimum RTT. The experiment tests 10%, 20%, 50%, and 80% thresholds with buffers ranging from 0.5× to 4× BDP.

The saved summaries compare QUIC-DC with CUBIC, BBRv2, NewReno, and Westwood+. Goodput, RTT, and loss are reviewed together because larger buffers can reduce drops while increasing delay.

3.4 PowerTCP Datacenter Experiment

PowerTCP was examined through supplied ns-3 datacenter configurations, scripts, logs, and flow-completion outputs. It uses a throughput-to-delay objective and queue feedback to adjust rates quickly. Since it targets datacenter traffic and uses a different simulator, its results are discussed separately from the Linux TCP tests.

3.5 DNCCQ-PPO Reinforcement-Learning Pipeline

DNCCQ-PPO combines Mininet, Redis telemetry, XQUIC, and Python training code. The intended loop sends network observations to a PPO agent, which returns rate or congestion-window actions to the QUIC implementation.

The topology, Open vSwitch, traffic shaping, and XQUIC endpoints started successfully. Redis showed bandwidth changes from 10 to 90 Mbps and RTT changes from 7 to 37 ms. However, the prebuilt XQUIC executable did not send the expected rlccstate_1001 messages, so the trainer received no observations and full PPO training was not verified.

4. RESULTS AND DISCUSSION

4.1 BBR vs CUBIC: Solo and Shared Bottleneck

Scenario	Bottleneck	Configured delay	BBR result	CUBIC result	Jain fairness	Interpretation
Solo baseline	10 Mbps	Single active flow	9.46 Mbps (BBR)	9.54 Mbps (CUBIC)	Not applicable	Retransmissions: 0 (BBR), 24 (CUBIC).
Condition A	10 Mbps	20 ms	1.86 Mbps	8.65 Mbps	0.643	CUBIC achieved about 4.6× BBR throughput.
Condition B	5 Mbps	50 ms	1.77 Mbps	3.82 Mbps	0.630	CUBIC achieved about 2.2× BBR throughput.

CUBIC received 8.65 Mbps versus BBR's 1.86 Mbps in the 10 Mbps test (fairness 0.643). In the 5 Mbps test, CUBIC received 3.82 Mbps versus BBR's 1.77 Mbps (fairness 0.630). These results show uneven sharing in the tested FIFO setup, not that CUBIC always outperforms BBR.

4.2 QUIC-DC and Baselines Across Buffer Sizes

Algorithm / configuration	Goodput (Mbps)	Average RTT (ms)	Loss (%)	Note
QUIC-DC (10%)	23.449	217.41	0.0083	Delay threshold controller
CUBIC	8.172	53.09	0.0155	—
BBRv2	24.966	249.44	42.7907	High measured loss in this configuration
NewReno	17.225	53.17	0.0084	—
Westwood+	23.449	217.02	0.0138	—

At 4×BDP, QUIC-DC recorded 23.449 Mbps goodput (about 93.8% of the 25 Mbps link) with 0.0083% loss in the 10% threshold row. BBRv2 reached 24.966 Mbps but had 42.7907% measured loss and 249.44 ms average RTT. This run shows why throughput should be considered alongside loss and delay.

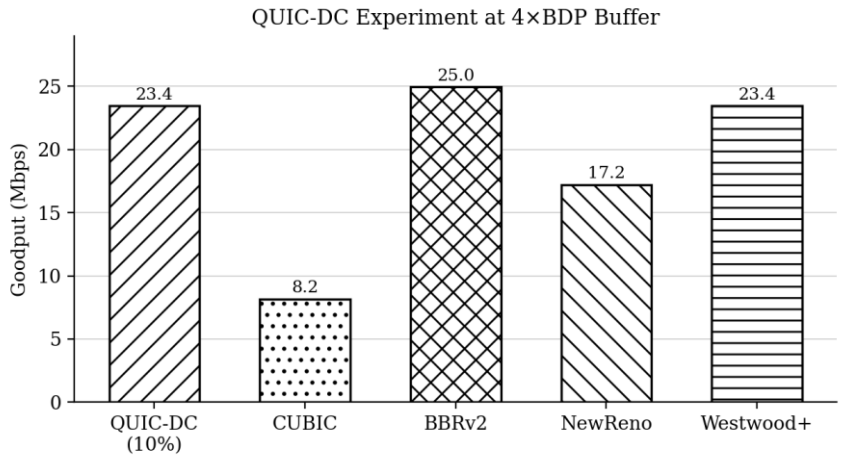


Figure 4.1. Goodput in the 4×BDP QUIC-DC experiment. Loss is reported separately in the preceding table because its scale differs substantially by algorithm.

4.3 TCP Reno vs Westwood+: Recorded Receiver Throughput

Algorithm	Receiver throughput per trial (Mbps)	Mean (Mbps)	Sample SD (Mbps)	Source status
TCP Reno	23.252, 23.254, 23.254	23.253	0.001	CHECK_REQUIRED
TCP Westwood+	16.410, 16.833, 16.864	16.702	0.253	CHECK_REQUIRED

Values in the table above are taken from the supplied validated-v2 CSV; the source status remains CHECK_REQUIRED. The three trials averaged 23.253 Mbps for Reno and 16.702 Mbps for Westwood+. Since the source flags byte-counter and algorithm-label problems, these figures need validation before drawing a wider conclusion.

4.4 PowerTCP: Datacenter-Oriented Evaluation

The presentation reports a Jain fairness index of 0.99994 across six flows, 1.153 ms average FCT in a web-search workload, and 39.4 KB average queue length. It also reports lower average and maximum FCT than HPCC in the stated comparison. These are presentation-reported ns-3 results and should be checked against the simulator outputs before formal publication.

Interpretation: The datacenter experiment targets a different operating regime: high link rates, microsecond-scale feedback, incast bursts, and short flow completion. Its most relevant metrics are fairness, queueing, and FCT rather than only bulk-transfer Mbps.

4.5 DNCCQ-PPO: Implementation Status and Finding

Aspect	Observed status	Conclusion
Environment and telemetry	Mininet, Open vSwitch, traffic shaping, and XQUIC started. Redis showed bandwidth changes from 10–90 Mbps and RTT from 7–37 ms.	Setup and telemetry verified.
RL state/action loop	The prebuilt XQUIC executable did not emit the expected rlccstate_1001 messages.	The trainer received no state; full PPO training was not verified.
Next step	Compile XQUIC with the custom RL hooks and validate state/action exchange.	Train PPO and compare it with fixed baselines.

4.6 Overall Comparison and Discussion

Algorithm	Strongest recorded evidence	Qualification / limitation
Reno	23.253 Mbps mean receiver throughput in the supplied Mininet test.	Preliminary only: counter reconciliation and algorithm labels require validation.
CUBIC	Strong share in the BBR coexistence cases; 8.65 Mbps vs 1.86 Mbps for BBR in Condition A.	Can fill shared FIFO queues and produce unfairness; result depends on setup.
BBR / BBRv2	Solo baseline near 9.46 Mbps in the 10 Mbps test; 24.966 Mbps in QUIC-DC 4×BDP summary.	Model-based pacing may compete poorly in shallow FIFO settings; a separate run showed high loss.
Westwood+	16.702 Mbps mean receiver throughput in the Reno experiment; 23.449 Mbps goodput in a distinct 4×BDP test.	Do not merge these different results; the testbeds differ and the Reno comparison is preliminary.
QUIC-DC	23.449 Mbps (about 93.8% of 25 Mbps) with low loss in the 4×BDP configuration.	Promising for delay/loss-sensitive flows in this experiment.
PowerTCP	Presentation reports 0.99994 fairness and low FCT under datacenter workload.	Datacenter-focused and measured in ns-3; not directly comparable with TCP namespace tests.
DNCCQ-PPO	Dynamic conditions and Redis telemetry validated.	Full RL control loop remains incomplete because expected state messages were absent.

4.7 Threats to Validity

- The algorithms used different tools, topologies, queues, and workloads, so results should not be ranked across unrelated testbeds.
- Some scenarios had few runs. Reno–Westwood+ logs also flag byte-counter mismatches and missing algorithm labels.
- Replication changes and settings such as RTT, buffer size, and queue discipline can affect results; DNCCQ-PPO did not reach a complete learning loop.

5. CONCLUSION AND FUTURE WORK

5.1 Conclusion

The BBR–CUBIC tests showed that similar solo throughput can become uneven when both flows share a bottleneck; CUBIC received more bandwidth in the tested FIFO conditions. QUIC-DC achieved high goodput with low measured loss in one tested configuration, while BBRv2's run showed high loss and RTT despite its goodput. The Reno–Westwood+ results remain preliminary, PowerTCP targets a different datacenter setting, and DNCCQ-PPO still needs a working state/action loop.

There is no single winner across all scenarios. Throughput, low delay, fairness, loss robustness, and flow completion time are different goals, so the test conditions and raw measurements must be reported clearly.

5.2 Future Work

- Use a common test matrix for compatible algorithms and repeat runs, reporting averages and variation.
- Validate Reno–Westwood+ by reconciling byte counts and confirming the active algorithm.
- Extend BBR–CUBIC tests to more RTTs, buffer sizes, competing flows, and queue-management methods.
- Complete XQUIC state/action integration, then train and evaluate PPO against fixed baselines.
- Archive consistent metric definitions, raw outputs, and scripts; verify PowerTCP values against ns-3 logs.

REFERENCES

[1] Y. Kuramo and S. Yamaguchi, “RTT Fairness Improvement in TCP BBR with Dynamic Target Adjustment.”

[2] W. Yu, J. Liu, X. Di, P. Xiao, and H. Qi, “DNCCQ-PPO: A dynamic network congestion control algorithm based on deep reinforcement learning for XQUIC.”

[3] V. Addanki, O. Michel, and S. Schmid, “PowerTCP: Pushing the Performance Limits of Datacenter Networks.”

[4] S. Mascolo, C. Casetti, M. Gerla, M. Y. Sanadidi, and R. Wang, “TCP Westwood: Bandwidth Estimation for Enhanced Transport over Wireless Links.”

[5] S. Mascolo, A. V. Balillo, G. Manfredi, D. D’Agostino, and L. De Cicco, “QUIC Delay Control: an implementation of congestion and delay control.”

[6] M. Hock, R. Bless, and M. Zitterbart, “Experimental Evaluation of BBR Congestion Control.”

[7] R. Ware, M. K. Mukerjee, S. Seshan, and J. Sherry, “Modeling BBR’s Interactions with Loss-Based Congestion Control.”

[8] B. Turkovic, F. A. Kuipers, and S. Uhlig, “Fifty Shades of Congestion Control: A Performance and Interactions Evaluation.”

[9] A. Abrol, P. M. Mohan, T. Truong-Huu, and M. Gurusamy, “BBR Congestion Control Algorithms: Evolution, Challenges and Future Directions.”

[10] F. B. Sarpkaya, A. Srivastava, F. Fund, and S. Panwar, “BBR’s Sharing Behavior with CUBIC and Reno.”

[11] A. Piotrowska, “Performance Evaluation of TCP BBRv3 in Networks with Multiple Round Trip Times.”

[12] D. Zeynali et al., “Promises and Potential of BBRv3.”

[13] A. Abrol, P. M. Mohan, and T. Truong-Huu, “FaiRTT: An Empirical Approach for Enhanced RTT Fairness and Bottleneck Throughput in BBR.”

[14] Z. Zhang, S. Li, Y. Ge, G. Xiong, Y. Zhang, and K. Xiong, “PBQ-Enhanced QUIC: QUIC with Deep Reinforcement Learning Congestion Control Mechanism.”

[15] K. Ono, D. Nobayashi, et al., “Performance Evaluation of Fairness Improvement Method between Different Types of Congestion Control by P4 BMv2.”

[16] L. A. Grieco and S. Mascolo, “Performance Evaluation and Comparison of Westwood+, New Reno, and Vegas TCP Congestion Control.”